

PRD — Personal AI Knowledge Assistant

Version: 1.0

Status: MVP / Development Ready

Product Type: Personal Knowledge Management + AI Assistant

Target Platform: Web

Primary Users: Individual users

Core Technology: Next.js / React + Supabase PostgreSQL + pgvector + LLM API

1. Product Overview

1.1 Nama Produk

Personal AI Knowledge Assistant

Nama sementara dapat diganti nantinya.

1.2 Deskripsi

Personal AI Knowledge Assistant adalah aplikasi berbasis AI yang memungkinkan pengguna menyimpan berbagai informasi pribadi seperti:

- catatan
- dokumentasi
- artikel
- materi belajar
- PDF
- TXT
- Markdown
- informasi pekerjaan
- dokumentasi project
- knowledge base pribadi

Pengguna kemudian dapat berinteraksi dengan seluruh data tersebut melalui interface chat menggunakan pendekatan **Retrieval-Augmented Generation (RAG)**.

Contoh:

User: "Bagaimana cara deployment project Laundry Hub yang pernah aku catat?"

AI akan:

1. memahami pertanyaan,

2. melakukan semantic search terhadap knowledge milik user,
 3. mengambil dokumen yang relevan,
 4. memasukkan context tersebut ke prompt LLM,
 5. menghasilkan jawaban,
 6. menampilkan sumber informasi yang digunakan.
-

2. Problem Statement

Informasi pribadi pengguna biasanya tersebar di berbagai tempat:

- Notion
- Google Docs
- file PDF
- catatan lokal
- dokumentasi project
- Markdown
- chat
- knowledge base pribadi

Masalahnya adalah pengguna kesulitan menemukan kembali informasi tersebut.

Traditional search memiliki keterbatasan.

Misalnya user memiliki catatan:

"Untuk deploy React ke VPS, gunakan Nginx sebagai reverse proxy..."

Kemudian user bertanya:

"Gimana cara publish frontend React ke server?"

Keyword search mungkin tidak menemukan dokumen tersebut karena kata **"publish frontend React"** berbeda dengan **"deploy React ke VPS"**.

Dengan semantic search + embeddings, sistem dapat memahami bahwa kedua konsep tersebut berkaitan.

3. Product Goals

Primary Goals

1. User dapat menyimpan knowledge pribadi.

2. User dapat meng-upload dokumen.
3. Sistem otomatis memproses dokumen menjadi embeddings.
4. User dapat bertanya melalui chat.
5. AI dapat menjawab berdasarkan knowledge milik user.
6. AI tidak boleh menggunakan knowledge milik user lain.
7. AI dapat menampilkan sumber jawaban.
8. Chat dapat menerima respons secara streaming.
9. Sistem memiliki arsitektur yang mudah dikembangkan.

Secondary Goals

- document categorization
 - tagging
 - conversation history
 - document management
 - knowledge search
 - AI summarization
 - multi-document reasoning
-

4. Non-Goals — MVP

Fitur berikut **tidak wajib** pada MVP:

- collaborative workspace
- team knowledge base
- realtime collaboration
- mobile application
- voice assistant
- image understanding
- OCR
- browser extension
- WhatsApp integration
- autonomous AI agent
- fine-tuning model

Fitur tersebut dapat masuk roadmap setelah MVP stabil.

5. Target User

Persona

Individual Developer / Student / Knowledge Worker

Contoh penggunaan:

Developer

Menyimpan:

Laravel documentation
Project documentation
Deployment notes
Database schema
API documentation
Troubleshooting notes

Kemudian bertanya:

"Kenapa deployment API-ku error?"

Student

Menyimpan:

Materi kuliah
PDF
Jurnal
Catatan
Modul

Kemudian:

"Jelaskan konsep normalisasi database dari materi yang aku upload."

Knowledge Worker

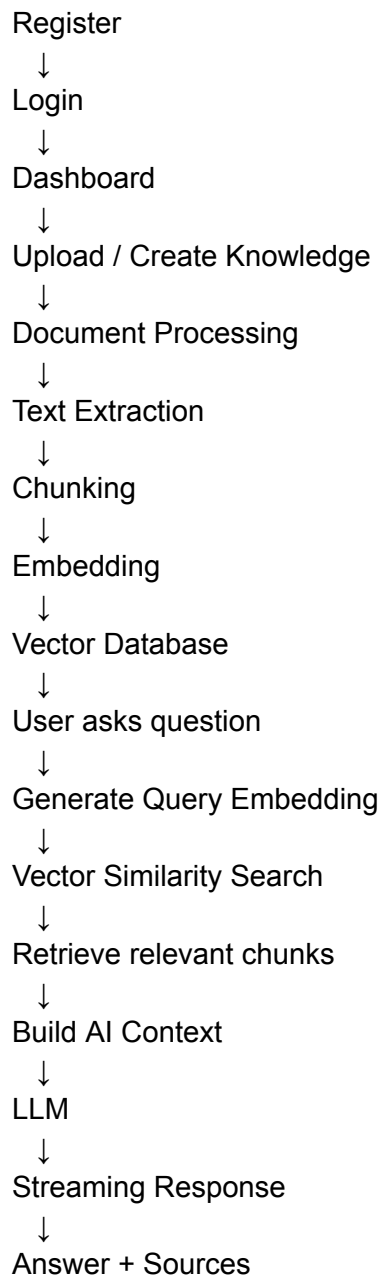
Menyimpan:

Meeting notes
SOP
Documentation
Research
Reports

Kemudian:

"Apa keputusan yang dibuat pada meeting minggu lalu?"

6. Core User Journey



7. Functional Requirements

7.1 Authentication

User dapat:

- Register
- Login

- Logout
- Reset password
- Update profile

Authentication menggunakan:

Supabase Auth

User memiliki unique:

user_id

yang menjadi dasar isolasi seluruh data.

8. Dashboard

Dashboard menampilkan:

Statistics

Total Documents

Total Knowledge

Total Conversations

Total Tokens Used

Storage Used

Contoh:

Personal AI Assistant			
Documents	Knowledge	Conversations	
24	1,240	38	
Ask your knowledge...			

9. Knowledge Management

User dapat membuat knowledge secara manual.

Create Knowledge

Field:

title
content
category
tags

Contoh:

Title:
Deploy React ke VPS

Category:
Development

Tags:
react, nginx, vps, deployment

Content:
Untuk melakukan deployment React...

10. Document Upload

User dapat upload:

MVP

- PDF
- TXT
- Markdown
- DOCX

Maximum file size misalnya:

10 MB

File disimpan di:

Supabase Storage

Metadata disimpan di PostgreSQL.

11. Document Processing Pipeline

Setelah upload:

Upload
↓
Create document record
↓
Extract text
↓
Normalize text
↓
Chunk text
↓
Generate embeddings
↓
Store chunks
↓
Mark document as READY

Document memiliki status:

UPLOADING
PROCESSING
READY
FAILED

12. Text Chunking

Dokumen tidak langsung dimasukkan seluruhnya ke embedding.

Contoh:

Document
↓
Chunk 1
Chunk 2
Chunk 3
Chunk 4
...

Recommended initial configuration:

chunk_size: 500–1000 tokens
chunk_overlap: 50–150 tokens

Nilai tersebut sebaiknya configurable.

13. Embedding

Setiap chunk dibuat menjadi vector.

Contoh konsep:

"Laravel menggunakan middleware..."

↓

Embedding Model

↓

[0.012, -0.823, 0.221, ...]

Vector disimpan menggunakan:

pgvector

14. Vector Search

Ketika user bertanya:

"Bagaimana authentication Laravel bekerja?"

System:

Question

↓

Embedding

↓

Vector similarity search

↓

Top K chunks

Contoh:

Chunk #12 → similarity 0.91

Chunk #44 → similarity 0.87

Chunk #08 → similarity 0.82

Chunk #31 → similarity 0.79

MVP dapat menggunakan:

Top K = 5

15. Critical Security Requirement

AI hanya boleh mengakses knowledge milik authenticated user.

Misalnya:

User A

- |— Document A1
- |— Document A2
- |— Document A3

User B

- |— Document B1
- |— Document B2

Ketika User A bertanya:

"What do I know about Laravel?"

retrieval hanya boleh mencari:

Document A1
Document A2
Document A3

Tidak boleh:

Document B1
Document B2

16. Row Level Security

Supabase PostgreSQL harus menggunakan **RLS**.

Contoh konsep policy:

`auth.uid() = user_id`

RLS harus diterapkan pada minimal:

documents
knowledge
document_chunks
conversations

messages

Jangan hanya mengandalkan filter:

WHERE user_id = ...

di application layer.

Security harus tetap enforced di database.

17. AI Chat

Halaman utama aplikasi adalah interface chat.

Contoh:

Personal AI Assistant	
User	Bagaimana cara deploy React ke VPS?
AI	Berdasarkan knowledge yang kamu simpan, deployment dapat dilakukan menggunakan...
Sources	└ VPS Deployment.md └ React Documentation.pdf
Ask anything about your knowledge... [➤]	

18. Streaming Response

Response AI harus ditampilkan secara streaming.

Bukan:

[waiting 8 seconds]

full response muncul

Tetapi:

Berdasarkan...

Berdasarkan knowledge...

Berdasarkan knowledge yang...

hingga selesai.

Teknologi dapat menggunakan:

SSE

atau mekanisme streaming API LLM yang tersedia.

19. Conversation

User dapat membuat conversation.

Conversation memiliki:

id

user_id

title

created_at

updated_at

Message:

id

conversation_id

role

content

created_at

Role:

system

user

assistant

20. Conversation History

Sidebar:

Today

- └─ Laravel deployment
- └─ Database optimization

Yesterday

- └─ Skripsi research
- └─ Supabase authentication

Previous

- └─ React architecture

User dapat membuka kembali conversation.

21. Source / Citation

Salah satu fitur penting adalah AI harus menunjukkan **dari mana jawabannya berasal**.

Contoh:

Berdasarkan dokumentasi deployment yang kamu simpan, aplikasi React dapat dijalankan menggunakan Nginx sebagai static file server.

Sources:

- [1] React VPS Deployment.md
- [2] Server Documentation.pdf

User dapat klik source untuk membuka document.

22. AI Prompt Strategy

LLM diberikan:

System Instructions

+

Retrieved Context

+

Conversation History

+

Current User Question

Konsep:

SYSTEM

You are a personal knowledge assistant.

Answer the user's question using the provided knowledge context.

If the answer cannot be found in the provided context, clearly state that the information was not found in the user's knowledge.

Do not fabricate information.

CONTEXT

[Document 1]

...

[Document 2]

...

CONVERSATION

User:

...

Assistant:

...

CURRENT QUESTION

...

23. Hallucination Control

AI harus memiliki aturan:

Jika informasi tersedia

Jawab berdasarkan context.

Jika informasi tidak tersedia

Jawab:

"Saya tidak menemukan informasi tersebut di knowledge base kamu."

AI **tidak boleh berpura-pura** bahwa informasi berasal dari knowledge user jika sebenarnya tidak ditemukan.

Namun sistem dapat menyediakan mode berbeda nantinya:

Knowledge Only

dan:

Knowledge + General AI

Untuk MVP, disarankan:

Knowledge Only

agar behaviour RAG lebih mudah diuji.

24. Search Modes

MVP:

Semantic Search

Menggunakan embedding similarity.

Future:

Hybrid Search

Gabungan:

Vector Search

+

Full Text Search

Future improvement:

Query

↓

Query rewriting

↓

Hybrid retrieval

↓

Reranking

↓

LLM

25. Database Schema

Minimal schema:

profiles
documents
document_chunks
knowledge
conversations
messages

profiles

id
email
full_name
avatar_url
created_at
updated_at

documents

id
user_id
title
file_name
file_path
mime_type
file_size
status
metadata
created_at
updated_at

document_chunks

id
document_id

user_id
content
chunk_index
embedding
metadata
created_at

Vector:

embedding vector(...)

Dimension mengikuti embedding model yang digunakan.

knowledge

id
user_id
title
content
category
tags
metadata
created_at
updated_at

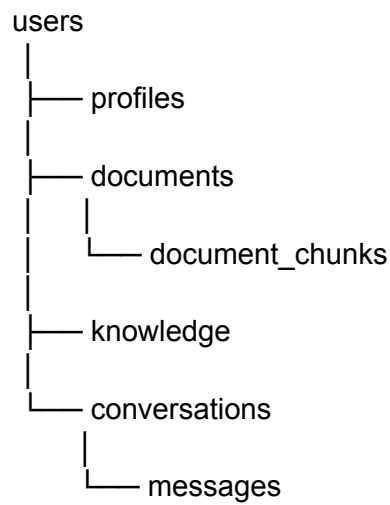
conversations

id
user_id
title
created_at
updated_at

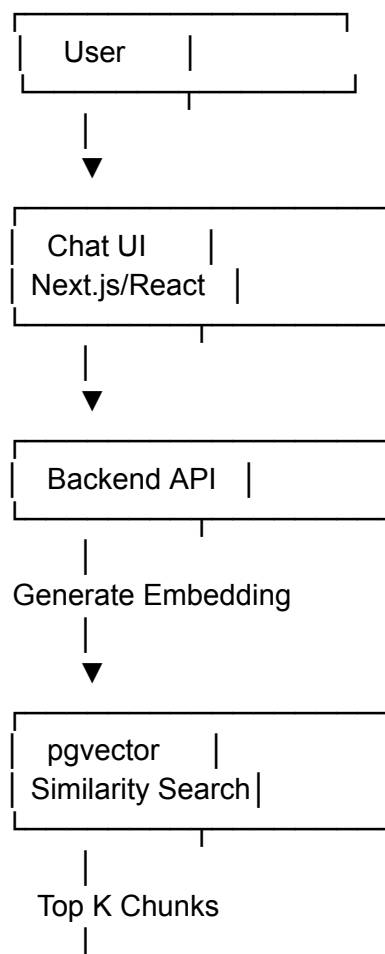
messages

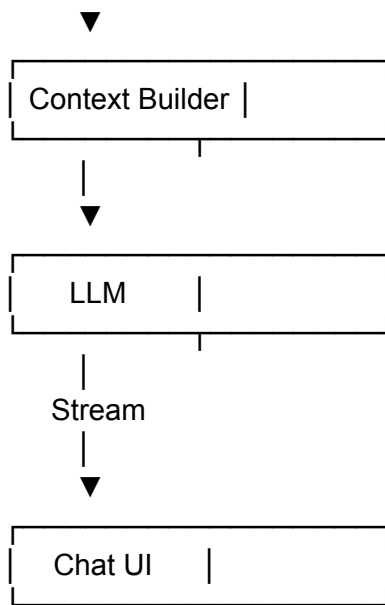
id
conversation_id
user_id
role
content
metadata
created_at

26. Recommended Database Relationships

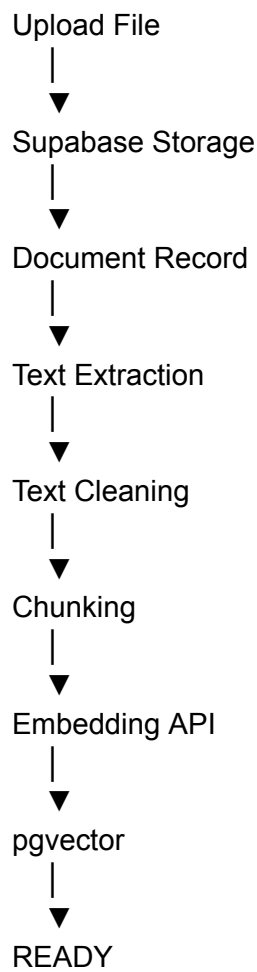


27. RAG Architecture





28. Document Ingestion Architecture



29. Backend Responsibilities

Backend bertanggung jawab atas:

- authentication verification
- document processing
- text extraction
- chunking
- embedding generation
- vector search
- RAG orchestration
- LLM communication
- streaming response
- conversation management
- authorization

LLM API key **tidak boleh berada di frontend**.

30. Suggested API

Authentication

Ditangani oleh Supabase Auth.

Knowledge

GET /api/knowledge
POST /api/knowledge
GET /api/knowledge/:id
PUT /api/knowledge/:id
DELETE /api/knowledge/:id

Documents

GET /api/documents
POST /api/documents
GET /api/documents/:id
DELETE /api/documents/:id

Chat

POST /api/chat

Request:

{

```
"conversation_id": "uuid",  
"message": "Bagaimana cara deploy Laravel?"  
}
```

Response:

streaming response

Conversations

GET /api/conversations

POST /api/conversations

GET /api/conversations/:id

DELETE /api/conversations/:id

31. Suggested Tech Stack

Frontend

Recommended:

Next.js

React

TypeScript

Tailwind CSS

UI:

shadcn/ui

Optional:

Framer Motion

Backend

Untuk MVP saya justru menyarankan **Next.js API / Route Handlers** terlebih dahulu daripada langsung membuat backend terpisah.

Next.js

├── Frontend

└── API

Jika project membesar, backend dapat dipisah.

Database

Supabase
PostgreSQL
pgvector
Supabase Storage
Supabase Auth

AI

LLM:

OpenAI

atau provider lain yang compatible.

Embedding:

Embedding API

Framework RAG:

LangChain

atau:

LlamaIndex

Recommendation

Untuk MVP, **jangan terlalu bergantung pada LangChain** jika pipeline-nya sederhana.

Bisa dibuat:

Embedding
↓
Supabase RPC
↓
Top K chunks
↓
Prompt
↓
LLM

Framework bisa ditambahkan ketika orchestration semakin kompleks.

32. Supabase RPC

Vector search sebaiknya dibuat sebagai PostgreSQL function.

Contoh konsep:

```
match_document_chunks(  
  query_embedding,  
  match_threshold,  
  match_count  
)
```

Function wajib mempertimbangkan:

user_id

sehingga retrieval tetap terisolasi.

33. Security Requirements

Authentication

Semua API private harus membutuhkan authenticated session.

Authorization

User hanya boleh:

READ own data
CREATE own data
UPDATE own data
DELETE own data

Database

Aktifkan:

RLS

Storage

File tidak boleh public secara default.

Gunakan:

private bucket

dan signed URL jika diperlukan.

API Keys

LLM API key hanya berada di server:

.env

Tidak boleh:

NEXT_PUBLIC_OPENAI_API_KEY

34. Error Handling

Document processing gagal:

status = FAILED

User mendapatkan:

"Dokumen gagal diproses."

Chat failure:

"Maaf, terjadi kesalahan ketika memproses pertanyaan."

Embedding failure:

- retry
- logging
- document tetap **PROCESSING** atau **FAILED**

LLM timeout:

- retry terbatas
 - tampilkan error friendly
-

35. Loading States

UI harus memiliki state:

Document

Uploading...
Processing...
Ready
Failed

Chat

Thinking...
Searching knowledge...
Generating...

Tidak perlu memperlihatkan proses internal secara literal pada MVP jika ingin UI lebih sederhana.

36. Empty States

Dashboard:

"Belum ada knowledge."

Chat:

"Mulai tanyakan sesuatu tentang knowledge kamu."

Documents:

"Belum ada dokumen. Upload dokumen pertamamu."

37. MVP Feature Scope

Must Have

- Register/Login
- Dashboard
- Create knowledge
- Edit knowledge
- Delete knowledge
- Upload document
- PDF/TXT/MD/DOCX processing
- Text extraction
- Chunking

- Embedding
 - pgvector
 - Semantic search
 - RAG
 - AI chat
 - Streaming response
 - Conversation history
 - Source citation
 - RLS
 - Private storage
-

38. Nice to Have

Setelah MVP:

- folders
 - tags
 - categories
 - favorites
 - search documents
 - regenerate response
 - edit message
 - delete conversation
 - rename conversation
 - markdown rendering
 - code syntax highlighting
 - export conversation
-

39. Future Features

Phase 2

Hybrid Search

Reranking

Better chunking

Query rewriting

Document summarization

AI-generated tags

Phase 3

Google Drive integration

Notion integration
GitHub integration
URL ingestion
Browser extension

Phase 4

Multi-user workspace
Shared knowledge base
Team permissions
Admin dashboard
Usage billing

40. Non-Functional Requirements

Performance

Target:

Initial chat response < 3 seconds

tergantung provider/model dan retrieval.

Streaming harus dimulai secepat mungkin.

Scalability

Architecture harus memungkinkan:

100 users
→ 1,000 users
→ 10,000 users

tanpa perubahan fundamental pada data model.

Availability

Jika AI provider mengalami masalah, aplikasi tetap dapat:

- login
 - melihat knowledge
 - melihat documents
 - melihat conversations
-

41. Observability

Backend harus mencatat:

request_id
user_id
conversation_id
retrieval_count
retrieval_latency
embedding_latency
llm_latency
total_latency
token_usage
error

Jangan menyimpan sensitive prompt/context secara sembarangan di production logs.

42. Cost Control

Karena setiap upload membutuhkan embedding dan setiap chat membutuhkan LLM:

System harus mencatat:

embedding tokens
input tokens
output tokens

MVP dapat memiliki:

daily usage limit

contoh:

50 chat messages/day

dan:

100 document uploads/month

Nilai final dapat dikonfigurasi.

43. Acceptance Criteria

Authentication

Given user belum login
When membuka halaman private
Then user diarahkan ke login.

Knowledge

Given user login
When membuat knowledge
Then knowledge hanya dapat dilihat oleh user tersebut.

Document

Given user upload PDF
When processing selesai
Then document memiliki status **READY** dan chunks tersedia.

Embedding

Given document telah diproses
When embedding selesai
Then setiap chunk memiliki vector embedding.

RAG

Given user memiliki document relevan
When user bertanya mengenai isi document
Then sistem mengambil relevant chunks dan menggunakan chunks tersebut sebagai context LLM.

Security

Given User A dan User B memiliki documents masing-masing
When User A melakukan query
Then retrieval tidak boleh mengembalikan chunk milik User B.

Chat

Given user mengirim pertanyaan
When LLM menghasilkan jawaban
Then jawaban ditampilkan secara streaming.

Sources

Given jawaban menggunakan document tertentu
When response selesai
Then UI menampilkan source document yang digunakan.

44. Development Milestones

Milestone 1 — Foundation

Next.js
Supabase
Authentication
Database
RLS
Basic UI

Output:

User sudah dapat register/login dan memiliki dashboard.

Milestone 2 — Knowledge

CRUD Knowledge
Document Upload
Storage
Document Metadata

Output:

User sudah bisa memasukkan knowledge.

Milestone 3 — RAG Pipeline

Text Extraction
Chunking
Embedding
pgvector
Similarity Search

Output:

Knowledge sudah dapat dicari secara semantic.

Milestone 4 — AI Chat

Conversation
Messages
Retrieval
Prompt
LLM
Streaming

Output:

User sudah dapat berbicara dengan knowledge base.

Milestone 5 — Source & Polish

Source citation
Loading states
Error handling
Markdown
Code highlighting
Responsive UI

Output:

MVP siap digunakan.

45. Definition of Done

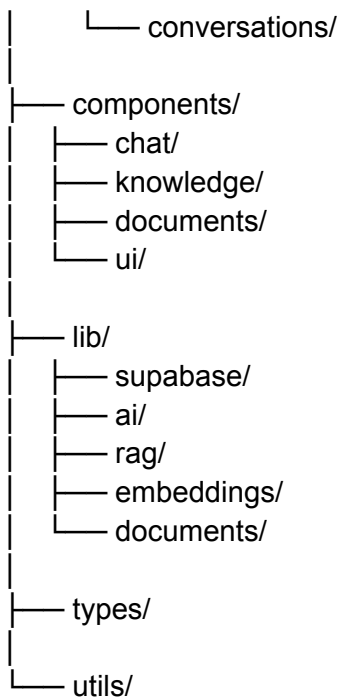
MVP dianggap selesai apabila:

- User dapat register/login.
 - User dapat membuat knowledge.
 - User dapat upload document.
 - Document berhasil diproses.
 - Document berhasil menjadi chunks.
 - Chunks memiliki embeddings.
 - Semantic search bekerja.
 - RAG menghasilkan jawaban berdasarkan knowledge.
 - Response AI streaming.
 - Source ditampilkan.
 - Conversation tersimpan.
 - RLS aktif.
 - User tidak dapat mengakses data user lain.
 - API key tidak terekspos ke browser.
 - Error handling tersedia.
 - UI responsive desktop/mobile.
-

46. Recommended Project Structure

Jika menggunakan Next.js:

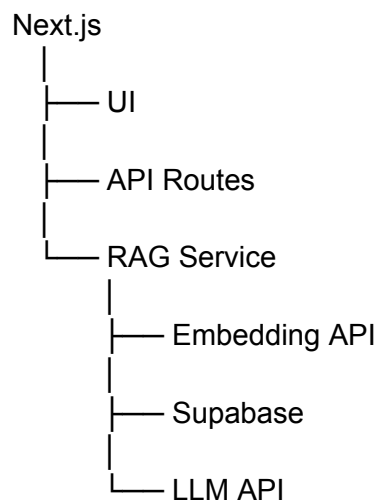
```
src/  
├── app/  
│   ├── (auth)/  
│   │   ├── login/  
│   │   └── register/  
│   ├── dashboard/  
│   ├── chat/  
│   │   └── [conversationId]/  
│   ├── knowledge/  
│   ├── documents/  
│   └── api/  
│       ├── chat/  
│       ├── knowledge/  
│       └── documents/
```



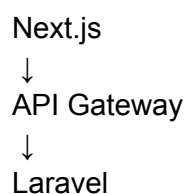
47. Important Architectural Decision

Saya menyarankan **jangan membuat sistem terlalu kompleks di awal**.

Untuk MVP:



Tidak perlu langsung:



↓
Python AI Service
↓
Redis
↓
RabbitMQ
↓
Vector DB
↓
LLM

Itu baru masuk akal ketika scale dan kompleksitas sudah meningkat.

Untuk **Personal AI Knowledge Assistant MVP**, PostgreSQL + pgvector + Supabase sudah sangat cukup.

48. Product Success Metrics

MVP dapat diukur menggunakan:

Retrieval Accuracy

% pertanyaan yang menemukan context relevan

Answer Groundedness

% jawaban yang benar-benar didukung knowledge

Response Time

Time To First Token

Total Response Time

User Engagement

questions/user/day

documents/user

returning users

System Reliability

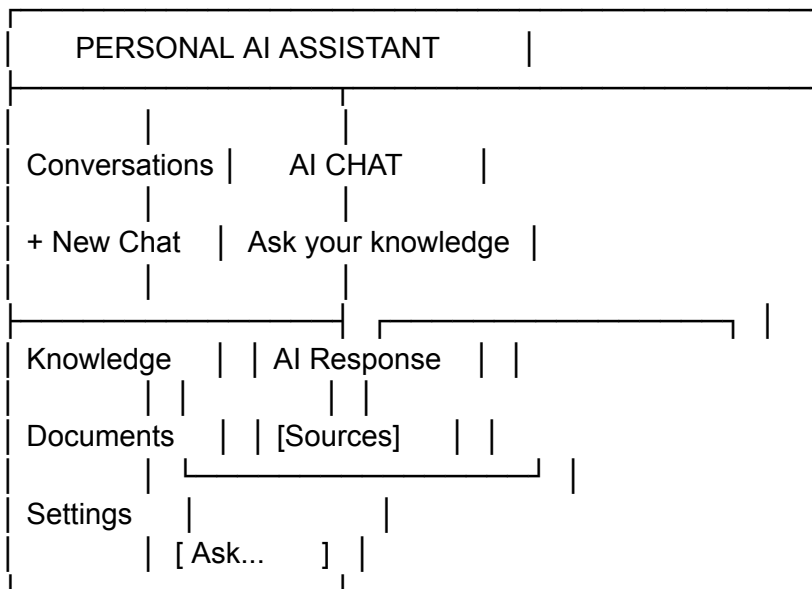
document processing success rate

chat success rate

embedding failure rate

49. MVP Final Scope

Kalau saya yang mengerjakan project ini, saya akan membatasi **versi pertama** menjadi:



Dengan backend:

Supabase Auth



PostgreSQL



Knowledge

Documents

Chunks

Conversations

Messages



pgvector



RAG Pipeline



LLM

Intinya, MVP ini jangan diposisikan sebagai "ChatGPT clone". Nilai utamanya adalah **"ChatGPT yang tahu knowledge pribadi user"**.

Dan menurut saya ini cukup bagus sebagai portfolio project karena dalam satu project kamu bisa menunjukkan **Next.js/React, PostgreSQL, Supabase, RLS, vector database, embeddings, RAG, LLM integration, streaming API, file processing, dan security** sekaligus.

